CS104-2

CS104-2

sudo bash bash bash

ways of running a script

Method	Runs in Current Shell	Needs `chmod +x`	Variables persist in current shell?	Requires Shebang
`source a.sh`	YES	NO	YES	NO
`./a.sh`	NO (child proc)	YES	NO	YES
`bash a.sh`	NO (child proc)	NO	NO	NO

- `echo -e` enables escape chars
- `\e` =esc char, `[` =starts ANSI seq
- `\e[31m` =red, `32m` =green, `34m` =blue, `0m` =reset
- `echo word1 word2` (tabs or extended spaces) are collapsed
  this is called *word splitting* ⇒**happens when " " are NOT used**
- `echo "word1 word2"` spaces are NOT collapsed becase "..." prevent word splitting

> word splitting also happens on unquoted variables

```
a="1  2  3"
for i in $a; do echo "$i"; done  # splits into 1, 2, 3
```

Variables are defined WITHOUT SPACES AROUND `=` operator as literal strings
`foo="abc def"; bar=1234` then

- `echo $foo$bar` ⟹ `abc def1234` (word splitting ✅)
- `echo "$foo$bar"` ⟹ `abc def1234` (word splitting ❌)
- `echo "${foo}XX${bar}` ⟹ `abc defXX1234` (`${...}` to explicitly seperate variable names)

Command subst: `$(..)` or `` `...` ``

Note: *without double quotes* bash does WORD SPLITTING and GLOB SUBSTITUTION
with double quotes None (but allows `$ ` !` special chars)
with single quotes everything is literal

`ls *.sh` ⇒ all files ending in .sh ; `ls "*.sh"` ⇒ the file named `*.sh`

Env vars

Print all using `env`

Variable	Description	Example Value
`$HOME`	Path to the current user's home directory	`/home/user`
`$USER`	Username of the currently logged-in user	`john_doe`
`$PATH`	Colon-separated list of directories for executable files	`/usr/local/bin:/usr/bin:/bin`
`$SHELL`	Path to the user's default shell	`/bin/bash`

Variable	Description	Example Value
`$PWD`	Present working directory	`/home/user/projects`
`$OLDPWD`	Previous working directory	`/home/user`
`$EDITOR`	Default text editor for the user	`vim` or `nano`
`$LANG`	System language and locale settings	`en_US.UTF-8`
`$TERM`	Terminal type (used by terminal emulators)	`xterm-256color`
`$UID`	User ID of the current user	`1000`
`$EUID`	Effective user ID (useful in scripts for checking root privileges)	`0` (if root)
`$PS1`	Primary prompt string (used for customizing shell prompt)	`\u@\h:\w\$`
`$PS2`	Secondary prompt string (used for multi-line commands)	`>`
`$IFS`	Internal field separator (controls word splitting)	`<space><tab><newline>`
`$?`	Exit status of the last executed command (0 = success, nonzero = failure)	`0` or `1`
`$$`	Process ID (PID) of the current shell instance	`12345`
`$!`	PID of the last background process	`6789`
`$0`	Name of the currently running script	`script.sh`
`$1, $2, ...`	Positional parameters (arguments passed to the script)	`arg1`, `arg2`, ...
`$#`	Number of arguments passed to the script	`2`
`$*`	All arguments as a single string	`"arg1 arg2 arg3"`
`$@`	All arguments as separate strings	`"arg1" "arg2" "arg3"`
`$RANDOM`	Generates a random number between 0 and 32767	`15862`
`$SECONDS`	Number of seconds since the shell started	`234`
`$LINENO`	Current line number in a script (useful for debugging)	`42`
`$OSTYPE`	Operating system type	`linux-gnu`
`$HOSTNAME`	System hostname	`my-computer`
`$DISPLAY`	X11 display server identifier (for GUI applications)	`:0`
`$XDG_SESSION_TYPE`	Current session type (e.g., X11 or Wayland)	`wayland`

Feature	`[ ]`	`[[ ]]`	`(( ))`
Remark	basic condnl (depr)	adv condl	arith evals
Int compar	`-eq, -lt, -gt`	`-eq, -lt, -gt`	`< > == !=`
Str compar	`!=, ==` (requires quoting)	`!=, ==, >, <, =~` (regex)	❌
Logical op	`-a (AND), -o (OR), !`	`&& || !`	`&& || !`
Word Splitting	YES	NO	NO
File testing	`-f, -d, -e`	`-f, -d, -e`	NO
Requires `$` prefix	YES	NO but preferred	NO
Supports `<=, =>`	NO only -le, -ge	NO only `> <` for str comp; but can use -le,-ge	YES numerically & cant use -lt,-le,...

Note

1. In `[[ ]]`, `>,<` works on *strings* so `[[ 10 > 2 ]] && echo "yes"` does not work
   Use `-gt, -lt` for numerical
2. Based on slides and some testing, inside `[[ ]]` or `(( ))`: `=` works same as `==`

More on arithmetic

- `let expr` evaluates arithmetic expr in scope

```
let "a = 5 + 4"
echo $a  # 9
let "a++"
echo $a  # 10
```

- `$[...]` is alt for `$((...))` ??? but deprecated
- `declare -i num` forces variable to integer type
- `num=7.5` gives "error token .5"
- `num="asdf"` sets it to 0

Difference Between `(( ))` and `$(( ))` in Bash

Both don't need $ prefixed variable names

`(( ))` = Arithmetic Evaluation then return 0/1 status

- returns 0 or **true** if result != 0 ; returns 1 or **false** if result == 0
- used with `if` statements
- the following does print `a is greater than b`

```
a=5
b=3
if (( a < b || ! (a > 10) )); then
    echo "a is greater than b"
fi

```

- also used for in-place modification w/o returning value
- the following does print `non-zero`

```
y=0
(( y+=5 )) && echo "non-zero"
```

`$(( ))` = Arithmetic Evaluation and result is returned

- used for returning exact values

```
a=3
b=5
result=$((a + b))  # 8
```

Floating-point

Bash does not support so need to use `bc` or `awk` as
`echo "3/4" | bc -l` prints `.75000000000000000000`

default scale (num of digits after `.` is 20)

supports internal variables as well as calling bash (external) variables with $ prefix

`echo "scale=3; a = 10; b = 3; a / b" | bc -l` prints `3.333`

By default, shell *truncates* after dec point so `4/3, 5/3` both evalute to `1` (say inside `let expr`)

On `bc` (basic calculator)

- In default mode, supports integer calculations
- `-l, --mathlib` Define the standard math library
- enables the use of `s(x)` , `c(x)` , `l(x)` , `e(x)` , and `a(x)` , which are **sin, cos, log, exp, and arctan**, respectively.
- also `sqrt(x)`
- sets precision to 20 (default) enabling floating point calculations
- `echo "(4 / 3 + 1) * 3" | bc` prints 6
- `echo "scale=3; (4 / 3 + 1) * 3" | bc` prints 6.999
- `echo "5 > 2" | bc` prints 1

In Shell

```
Success == exit code 0 == TRUE
Failure == exit code 1 == FALSE
```

applies to both commands and condl expressions like `((...)), [...],...`

control flow and loops

String:
`-z` empty ; `-n` non-empty
File:
`-e` exists ; `-f` is file ; `-d` is directory ; `-s` is non empty ; `-r/w/x` is r/w/x

> Status code == 0 evals to TRUE but arithmetic 0 evals to FALSE

...

```bash
# \[\[ ]] for string and/or numeric expressions

if [[ -f file.txt ]]; then
    echo "its a file"
elif [[ -d file.txt ]]; then
    echo "its a directory"
else
    echo "its not a file or directory"
fi

# > works lexically!!! can be replaced with -gt
# NOTE: >= <= will NOT work in \[\[ ]]

if [[ 3 > 1 && (-z $someUndefinedVarASDF) ]]; then
    echo "should run"
fi

# (( )) for numeric expressions (non zero evals to true)

if ls non_existent_directory &>/dev/null; then
    echo "Printed if successful ie status code = 0"
else
    echo "Printed if fails ie status code != 0"
fi

some_command
if [[ $? ]]; then
# this is bad tho, prefer:
# if [[ $? -ne 0 ]]; then
# if some_command; then

    echo "Printed if fails ie status code != 0"
else
    echo "Printed if successful ie status code = 0"
fi
```

⚠ **Notice the lack of parentheses in bash** `for`

```bash
for i in foo bar baz; do
    echo "$i"  # prints foo\nbar\nbaz
done

for i in "foo bar baz"; do
    echo "$i"  # prints foo bar baz (single element)
done

for i in {1..5..2}; do
    echo "$i"  # prints 1\n3\n5 (start stop step, inclusive)
done

for i in "${arr[@]}"; do
    echo "$i"  # prints each el on new line (" disables word splitting)
done
```

```
a=5
while (( a < 7 )); do echo $((a++)); done
# prints 5 6 8 ; finally, a=7

b=5
until (( b > 7 )); do echo $((b++)); done
# prints 5 6 7 ; finally, b=8
```

Special Vars (for script and/or functions)

`$0` name of script
`$1, $2...` posnl arguments
`$#` num of args
`$@` all args separately (enclose in double quotes and iterate over it to get `$1, $2...`)
`$*` all args as one string
`$?` last exit status code
`$$` shell's PID
`$!` PID of last bg task
`$_` last arg of last command

FUnctions

`return n` to return status code
`exit n` to exit script with status code
For capturing/reusing/returning meaningful output stick to stdout (`echo`)

```
factorial() {
    if [[ $1 -le 1 ]]; then
        echo 1
    else
        local temp=$(( $1 - 1 ))
        local result=$(factorial "$temp")
        echo $(( $1 * result ))
    fi
}
```

`local` to create variable scoped to immediate fn scope
default variables are global
`export` to allow usage in child processes (like ENV vars are always usable in all child procs)

Reading input

```
while read -r line; do   # reads raw text into variable line
    echo "Line: $line"
done < "$file"           # redirects contents of $file into the above

# alternatively for mentioning it in script use:
done <<EOF
This is Line 1
This is Line 2
EOF
```

Read options

Option	Meaning
`-r`	Don't interpret backslashes (`\`) as escape characters
`-p`	Prompt before reading input
`-a`	Read into an array
`-n N`	Read N characters only
`-s`	Silent input (e.g., for passwords)
`-t N`	Timeout after N seconds
`-u FD`	Read from file descriptor FD

Use `IFS= read -r line` to prevent word splitting / trimming for reading files as it is

heredocs and herestrings

create heredocs with `<<SYMBOL`

```
cat <<"EOF" > file
hello
world
EOF
```

quoting EOF disables var expansion inside heredoc

create herestrings with `<< "string $someVar"`

```
cat <<< "a singular string" > file
```

GOATED SHIT: Reading separate fields/words into an array
`IFS=',' read -ar arr <<< "apple,orange,banana"` reads them individually as elements into array `arr`

Note:

`printf "%-10s %s %.2f %x\n" "str1" "str2" "3.156" "32"`
prints `str1 str2 3.16 20`

`%d` : decimal
`%x` : hex

Syntax	Meaning	Example
`%5s`	Right-align string in 5 spaces	`"abc"` → `" abc"`
`%-5s`	Left-align	`"abc"` → `"abc "`
`%04d`	Pad integer with leading zeros	`5` → `0005`
`%.2f`	2 decimal places (rounding)	`3.14159` → `3.14`

- `printf` prints one string for each argument set passed!

```
printf "Line: %d\n" {1..3} > file.txt

name="John"
greeting=$(printf "Hello, %s!" "$name")
```

Arrays (read from slides thats it?)

```
arr=(foo bar baz "ohio rizz")  # 0 indexed

echo "${arr[1]}"  # bar
echo "${arr[@]}"      # all elements separately
echo "${#arr[@]}"     # number of elements

arr[2]="orange"

# slicing bash 4+
# arr[@]:start:length
echo "${arr[@]:1:2}"  # bar orange
echo "${arr[@]: -3: 2}"  # bar orange

unset 'arr[1]'  # delete
```

Assoc Arrays (Bash 4+)

```
declare -A capitals
capitals[India]="New Delhi"
capitals[France]="Paris"

echo "${capitals[India]}"

echo "${!capitals[@]}"    # all keys
echo "${capitals[@]}"    # all values

for country in "${!capitals[@]}"; do
  echo "$country => ${capitals[$country]}"
done

```

Shell Globbing

basic

Pattern	Matches
`*`	Matches **zero or more** characters (except `/` in pathnames)
`?`	Matches **exactly one** character (except `/`)
`[abc]`	Matches **one** character from the set `a`, `b`, or `c`
`[a-z]`	Matches any **single character** in the range `a` to `z`
`[^abc]`	Matches a character **not** in the set
`[!abc]`	Same as `[^abc]` (negation)

extended (works for me on bash 5.1)

requires enabling as `shopt -s extglob`

Pattern	Meaning
`?(pattern)`	Matches **zero or one** occurrence
`*(pattern)`	Matches **zero or more** occurrences
`+(pattern)`	Matches **one or more** occurrences
`@(pattern)`	Matches **exactly one** occurrence
`!(pattern)`	Matches anything **except** the pattern

Extended Patterns

Pattern	Meaning
`?(pattern)`	Matches **zero or one** occurrence
`*(pattern)`	Matches **zero or more** occurrences
`+(pattern)`	Matches **one or more** occurrences
`@(pattern)`	Matches **exactly one** occurrence
`!(pattern)`	Matches anything **except** the pattern

> You can use `|` to separate alternatives inside the parentheses.

You can use glob patterns in **variable substitution** to manipulate strings (not files).

Example 1: Remove Prefix or Suffix

```
str="report_final_version.txt"

# Remove shortest match from end
echo "${str%_*}"   # report_final

# Remove longest match from end
echo "${str%%_*}"  # report

# Remove shortest match from start
echo "${str#*_}"   # final_version.txt

# Remove longest match from start
echo "${str##*_}"  # txt
```

Here, `*` is used as a glob — it matches as many or as few characters as possible.

Bash has options to control how globs behave:

1. `shopt -s nullglob`
   - If no files match a glob, the glob **expands to nothing**.
   - Useful to avoid literal `*.txt` appearing in output.
2. `shopt -s failglob`
   - If no files match, the shell gives an **error**.
3. `shopt -s dotglob`

- includes **dotfiles** (like `.bashrc`) in glob matches.

4. `shopt -s globstar`
   - Enables `**` to match directories recursively.
   - `echo **/*.txt # all .txt files in current and subdirs`

Bash Regex (3+) in [[=~ REGEX]]

1. either leave it *unquoted* (quoting causes shell to treat as literal)
2. or define it as variable and use that
3. Bash supports ERE (like `egrep`)
4. whole match is stored in `${BASH_REMATCH[0]}`
5. capturing groups in `${BASH_REMATCH[1]}, ${BASH_REMATCH[2]}, ...`

```
email="foo@example.com"
if [[ $email =~ ([a-z]+)@([a-z]+\.[a-z]+) ]]; then
  echo "user: ${BASH_REMATCH[1]}"
  echo "domain: ${BASH_REMATCH[2]}"
fi

# output
# user: foo
# domain: example.com


date="2025-04-25"
regex='^([0-9]{4})-([0-9]{2})-([0-9]{2})$'
if [[ $date =~ $regex ]]; then
  echo "Year: ${BASH_REMATCH[1]}"
  echo "Month: ${BASH_REMATCH[2]}"
  echo "Day: ${BASH_REMATCH[3]}"
fi

```

Adv String Manipulation

1. String Length

To get the length of a string:

```
str="Hello, World!"
echo ${#str}  # Outputs: 13
```

---

2. Substring Extraction

Extract a substring using `${string:start:length}`:

```
str="Hello, World!"
echo ${str:7:5}  # Outputs: World
echo ${str:7}    # Outputs: World!
```

- `start`: Position to begin extraction (0-indexed).
- `length`: Number of characters to extract (optional).

3. String Replacement

- **First occurrence replacement**: `${string/old/new}`

```
str="Hello, World!"
echo ${str/World/Bash}  # Outputs: Hello, Bash!
```

- **Global replacement**: `${string//old/new}`

```
str="Hello, World! World!"
echo ${str//World/Bash}  # Outputs: Hello, Bash! Bash!
```

- **Using patterns (wildcard)**:

```
str="hello123world"
echo ${str/hello*/Hi}  # Outputs: Hi
```

---

4. String Trimming

- **Trim from the beginning**:

```
str="   Hello   "
echo ${str#"${str%%[![:space:]]*}"}  # Removes leading whitespace
```

- **Trim from the end**:

```
str="   Hello   "
echo ${str%"${str##*[![:space:]]}"}  # Removes trailing whitespace
```

- **Trim both sides**:

```
str="   Hello   "
echo ${str#"${str%%[![:space:]]*}"}  # Leading trim
echo ${str%"${str##*[![:space:]]}"}  # Trailing trim
```

---

5. Case Conversion

- **To uppercase**:

```
str="hello"
echo ${str^^}  # Outputs: HELLO
```

- **To lowercase**:

```bash
str="HELLO"
echo ${str,,}  # Outputs: hello
```

- **To title case (first letter upper, rest lower)**:

```bash
str="hello world"
echo ${str^}   # Outputs: Hello world
```

---

6. String Splitting

You can split a string into an array by using a delimiter:

```bash
str="apple,orange,banana"
IFS=',' read -ra arr <<< "$str"
echo ${arr[0]}  # Outputs: apple
echo ${arr[1]}  # Outputs: orange
```

---

7. String Substitution with `sed` or `awk`

For more complex substitutions, `sed` or `awk` can be used:

```bash
str="apple pie"
echo "$str" | sed 's/apple/orange/'  # Outputs: orange pie
```

---

8. Using `printf` for Formatting

`printf` allows formatted output:

```bash
printf "%-10s %-5d %.2f\n" "Name" 42 3.1416
# Outputs:
# Name       42   3.14
```

- `%-10s` : Left-aligns a string in a 10-character field.
- `%-5d` : Prints an integer in a 5-character field.
- `%.2f` : Prints a float with 2 decimal places.

This covers the essential Bash string manipulation techniques, helping you handle strings effectively in scripts.

sed

Below is a distilled, exam-ready sed guide that pulls in **every sed command** shown in your PDF (delete, substitute, print, append/insert/replace, quit, multiple commands, back-references, in-place, scripts), plus a suite of **non-trivial addressing and pattern-matching recipes** (last line, second-last line, last occurrence on a line, negation ranges, etc.). We omit deep hold-space tricks, focusing instead on versatile one-liners.

Invocation & Flags

```
sed [OPTIONS]… [SCRIPT] [INPUT…]
```

Flag	Meaning
`-e script`	add `script`
`-f file`	load script from `file`
`-n`	suppress automatic printing
`-i[SUFFIX]`	edit files in-place (optional backup `SUFFIX`)
`-E` / `-r`	enable extended regex

1. Addressing Lines

Address	Effect
`N`	line number `N`
`M,N`	lines `M` through `N`
`/pat/`	any line matching `/pat/`
`/start/,/end/`	range from first `/start/` to next `/end/`
`/pat/!`	any line **not** matching `/pat/`
`$`	last line
`$-1` (via workaround)	second-last line (see example below)

Examples

```
# delete lines 1–5, then print only non-empty lines
sed -n '1,5d; /^$/!p' file.txt

# keep only lines that do NOT match “ERROR”
sed '/ERROR/!d' logfile

# print only between markers (inclusive)
sed -n '/^BEGIN/,/^END/p' file.txt

# delete last line
sed '$d' file.txt

# print last line
sed -n '$p' file.txt

# print second-last line (pairwise read):
sed -n 'N;$!P;$p' file.txt

# delete last two lines
sed -e :a -e '$d;N;2,2ba' -e 'P;D' file.txt
```

2. Delete (`d`)

```
# line 6
sed '6d' file.txt

# lines 1–5
sed '1,5d' file.txt

# blank lines
sed '/^$/d' file.txt
```

---

3. Substitute (`s/pat/repl/[flags]`)

Flag	Meaning
`n`	replace only the nth occurrence on each line
`g`	global (all occurrences)
`p`	print the line if a substitution was made
`I`	case-insensitive

Basic examples

```
# swap “wolf” → “fox”
sed 's/wolf/fox/' bigfile

# only on line 3
sed '3s/wolf/fox/' bigfile

# 3rd occurrence, case-insensitive, everywhere
sed 's/wolf/fox/3gI' bigfile

# wrap each capital letter in parentheses
echo "Welcome To CS104" \
  | sed 's/\(\b[A-Z]\)/(\1)/g'
```

---

4. Print (`p`)

```
# line 1 (also prints twice unless -n)
sed '1p' bigfile

# only matching “scream”, with -n
sed -n '/scream/p' bigfile
```

---

5. Append / Insert / Replace

```
# after line 2
sed '2a tomato' fruits.txt

# before line 2
sed '2i tomato' fruits.txt

# change line 2 entirely
sed '2c aam' fruits.txt

# change every line matching /mango/
sed '/mango/c aam' fruits.txt
```

---

6. Quit Early (`q`)

```
# print first 5 lines then exit
sed '5q' fruits.txt
```

---

7. Multiple Commands

```
# semicolon-separated
sed 's/mango/aam/; s/banana/kela/' fruits.txt

# or via multiple -e
sed -e 's/mango/aam/' -e 's/banana/kela/' fruits.txt

# order matters!
echo "please fix the light bulb!" \
  | sed 's/light/tube/g; s/bulb/light/g'
```

---

8. Back-References & `&`

```
# replace double letters (AA, BB, etc.) with XX
echo "AA BB CC" \
  | sed 's/\([A-Z]\)\1/XX/g'

# wrap each number in [ ]
echo "I have 5 apples and 3 bananas." \
  | sed 's/[0-9]\+/[&]/g'
```

---

9. In-Place Editing & Scripts

```
# edit files directly, backup to .bak
sed -i.bak 's/DEBUG/INFO/g' *.log

# run from a script file
sed -f commands.sed file.txt
```

---

10. Advanced Pattern-Matching Tricks

a) Replace only the **last** occurrence of "foo" on each line

```
sed 's/\(.*\)foo/\1bar/' file.txt
```

b) Replace the **second-last** occurrence:

```
sed -E 's/(.*)(foo)(.*)(foo)/\1\3bar/' file.txt
```

c) Delete everything **before** the last comma on each line

```
sed 's/.*,//' file.csv
```

d) Match the **last** field of a colon-delimited record

```
sed -E 's/.*:([^:]+)/\1/' /etc/passwd
```

e) Print only the **second-last** line (another approach)

```
sed -n '1h;1!H;${;g;s/\n.*\n$/\n/;p}' file.txt
```

f) Invert a range: delete lines **outside** 10–20

```
sed '10,20!d' file.txt
```

g) Stop at the **first** match of "ERROR" and quit

```
sed '/ERROR/{p;q;}' logfile
```

---

11. Exam Tips

- **Always quote** your scripts (`'…'`) to avoid shell meta-character expansion.
- Combine `-n` with explicit `p` to control exactly what prints.
- Use `$` for "last line" and pattern ranges `/start/,/end/` .
- Greedy regex (`.*`) lets you isolate the **last** match on a line.
- Back-references (`\1` , `\2` , …) and `&` make complex replacements concise.
- For second-last/other "from-end" tasks, pairwise `N;P;p` or the accumulate-all (`h/H` + `g`) idiom works.

Keep this sheet beside you during exams: it covers **every** sed command from your PDF plus the non-trivial addressing/pattern examples you'll need!

awk

Reading input w `getline`

`fruits` has 4 lines (input is only 3 lines so last one is repeated)

```
$ awk '
{
getline line < "-"
print "You typed: " line " (" length(line) ")"
}' <<EOF
tooez
ggs
fly
EOF
You typed: tooez (5)
You typed: ggs (3)
You typed: fly (3)
You typed: fly (3)
```

`getline [var] [< file]`

reads a line from file (by default the current input file) into var (or $0 if not specified)



Reading input from a command

Can be done via `<command> | getline <var to store it in>`

- Note, this only reads the first line of input.
- the next time u run the same statment, it reads the next line!
- to prevent this run `close(<command>)` to close the buffer or whatever
- `getline` returns
  - `1` if a record is successfully read
  - `0` if end of file (EOF) is reached
  - `-1` on error
- so a typical idiom is `while((cmd | getline) > 0) { ... }`
- Note, same happens when you do `getline line < "<filepath>" or "-" for stdout`
- run `close(<filename>)` to prevent reading from next line (it will reset to first line always)

String manipulation and more

> **NOTE**
>
> 1. strings are **1-indexed**
> 2. regex matching (usually) returns ***longest, leftmost match***

- `length(string)`
- `substr(string, start[, len])`
- `!~` for negating regex match
- `gensub(regexp, replacement, how [, target])` returns substituted string
  `how = "g"` means global, else pass number >= 1
  `target` is `$0` by default
- `index(in, find)` ret 0 if not found
- `match(string, regexp[, array])` clears array and `arr[0]` full match and subsequent elements give matched groups
  - NOTE: for regexp as strings here, remember you have to **ESCAPE TWICE** so to match a literal `[` u need regex `\[` and hence use `\\[` in the definition of the regex pattern as a string
- `tolower/toupper`
- `split(string, array, delimiter)` splits the `string` into an array based on the `delimiter` (which defaults to whitespace). It returns the number of elements in the resulting array.
- `patsplit(string, array, pattern)` uses pattern instead of fixed delimiter
- `fflush([filename])` flush the buffer (stdout by default)
- `$0` gives whole record by default but running `$1=$1` forces it to reprocess and `$0` is rejoined using OFS instead of FS (which it was earlier)
- There is an important difference between the two cases of 'FS = " "' (a single space) and 'FS = "[\t\n]+"' (a regular expression matching one or more spaces, TABs, or newlines). For both values of FS, fields are separated by *runs* (multiple adjacent occurrences) of spaces, TABs, and/or newlines. However, when the value of FS is " ", awk first strips leading and trailing whitespace from the record and then decides where the fields are. For example, the following pipeline prints 'b':

  ```
  $ echo ' a b c d ' | awk '{ print $2 }'
  -| b
  ```

  However, this pipeline prints 'a' (note the extra spaces around each letter):

  ```
  $ echo ' a  b  c  d ' | awk 'BEGIN { FS = "[ \t\n]+" }
  >                           { print $2 }'
  -| a
  ```

- `FS == " "`
  Fields are separated by runs of whitespace. Leading and trailing whitespace are ignored. This is the default.
- `FS == any other single character`
  Fields are separated by each occurrence of the character. Multiple successive occurrences delimit empty fields, as do leading and trailing occurrences. The character can even be a regexp metacharacter; it does not need to be escaped.
- `FS == regexp`
  Fields are separated by occurrences of characters that match regexp. Leading and trailing matches of regexp delimit empty fields.
- `FS == ""`
  Each individual character in the record becomes a separate field. (This is a common extension; it is not specified by the POSIX standard.)

git: what cs guys think trees actually look like

> HEAD = latest commit of branch we are on
> (essentially it indicates the end of a *chronological* sequence of *related* commits)
>
> it is detached when we checkout to non-latest commit of some branch, or to some remote ref like `origin/main`
> and any commits made now are floating = **lost unless branched off**

Config

```
git config --global user.name/email "asdf"/"asdf@clg.edu.in"
git config --global core.editor "vim"
git config --list  # show all setting
git config user.name   # show specific setting
```

Adding

`git add .` stage all changes in cd
`git add -u` stage only modified and deleted *tracked* files in entire repo
Note, it does not add untracked files

`git add -A` stage everything everywhere
Does include untracked files

m/d/u = modification/deletion/add(was untracked earlier)

`git restore --staged <file>` to remove a file's m/d/u from staging area

`git commit -a ...` is the same as running `git add -u` and then committing

`git commit --amend` to append the staged changes to prev commit and amend commitmsggi

Log

`git log` shows commit history of current branch
`git log BRANCH`
`git log FILE` only of commits that touched file FILE
`git log --oneline` one line per commit: hash and commitmsg
`git log -p` shows consecutive commit diffs
`git log -n N` only last N commits
`git log --stat` shows summary of file changes per commit

`git log --graph --decorate --all` visualize commit history across all branches (add `--oneline` to easy reading)

`git log --since="2025-01-01"`

Diff

> diff output is a series of commands to change file a to file b

`git diff <COMMIT-A> [<COMMIT-B>]` (if left blank COMMIT-B is working directory) shows ***diff of B wrt A***
output:

```
                    (refers to commits a and b in order)
diff --git a/foo b/foo
                    (hashes computed for commits a and b in order)
index a2441b8..c92fafb 100644
                    (means we are showing diff of b/foo wrt a/foo)
--- a/foo
+++ b/foo
```

eg:
foo

```
you are all i need...
im in the middle of your picture...
lying in your reeds

its all wrong
```

boo

```
you are all i need
im in the middle of your picture...

its all wrong
bye
bye
```

`diff foo boo`

```
1c1
< you are all i need...
---
> you are all i need
3d2
< lying in your reeds
5a5,6
> bye
> bye
```

`1c1` : change line 1 in a to line 1 in b followed by `< <line-from-a>` and `> <line-from-b>`

`3d2` : delete line 3 in a and now it'll become line 2 in b followed by `< <line-from-foo>`

`5a5,6` : add lines 5,6 of b after line 5 of a followed by `> <line-from-b>`

`diff -u foo boo` (gives unified diff which is what git uses)

```
--- foo 2025-04-08 16:26:41.095857600 +0530
+++ boo 2025-04-08 16:43:33.781042900 +0530
@@ -1,5 +1,6 @@
-you are all i need...
+you are all i need
 im in the middle of your picture...
-lying in your reeds

 its all wrong
+bye
+bye
```

`--- file a` and `+++ file b`
`@@ -1,5 +1,6 @@` means diff will show a piece of text spanning lines 1-5 in foo and 1-6 in boo
`-` means it was deleted in b (as compared to a) *delete from a to make it like b*
`+` means it was added in b (as compared to a) *add to a to make it like b*

`git diff --cached <a>` shows staged changes wrt a

`git diff HEAD` shows changes in working directory (staged or unstaged) wrt latest commit of current branch ie HEAD

`git diff <commit> -- file-paths` to only diff some files

`git show COMMIT` detailed info
`git show :FILE` show file contents in *staging area*
`git show COMMIT:FILE` shows file contents in that commit
`git show BRANCH` latest commit

checkout

`git checkout COMMIT` rollback to previous commit in working dir (useful for only viewing)
`git checkout COMMIT FILE` rollback to previous commit in working dir (useful for only viewing)
alternate for above: `git restore --source=COMMIT <file>`

branches

`git branch [-r] [-a]` view local branches / remote branches / all
`git branch BRANCH` create new branch
`git switch BRANCH` checkout to it
`git switch -c BRANCH` create and checkout (useful if you're in detached head but u want to save changes to a new branch)
`git branch -b BRANCH` (same as above)
`git branch -d BRANCH` delete a branch (use `-D` to force)
`git branch -m OLDNAME NEWNAME`

`git merge -m "MERGE_COMMIT_MSG" branch-to-merge`
`git merge --no-ff branch` forces a merge commit even if a fast-forward is possible — useful for preserving merge history.

reverting

`git revert COMMIT` introduces new commit with reverse changes

⚡ **reset is dangerous**

```
git reset HEAD FILE      # unstage a file
git reset --soft HEAD~1  # undo last commit, keep changes staged
git reset --mixed HEAD~1 # undo last commit, keep changes in working directory
git reset --hard HEAD~1  # discard commit AND changes
git reset --hard COMMIT  # move to any commit (dangerous if not backed up)
```

`git reset HEAD` unstage changes (move them back to working directory) from the last commit
`git reset [--hard] HEAD~1` To discard the last commit (use with caution, as it rewrites history) (without brings reverted changes into working directory, hard removes them as well)
`git reset --hard ghi78` To move the branch pointer to a specific commit (e.g., ghi789)

Python

f string formatting

```python
# DEBUG PRINTING++
print(f'Debugging {bugs=} {count=} {area=}')
# Debugging bugs='roaches' count=13 area='living room'

# STRING PADDING
for name, phone in table.items():
    print(f'{name:10} ==> {phone:10d}')

# Sjoerd     ==>       4127
# Jack       ==>       4098
# Dcab       ==>       7678
```

builtins

```python
abs(x), round(x[, ndigits]), pow(x, y[, mod])
divmod(a, b)                    # (a // b, a % b)
bin(x), oct(x), hex(x), int(x)
all(iterable), any(iterable)
enumerate(iterable[, start=0])
filter(func, iterable), map(func, iterable)
sum(iterable[, start]), max(iter), min(iter), len(iter)
reversed(seq), sorted(iterable, key=..., reverse=...)
zip(*iterables)
type(x), isinstance(x, type)
id(x), hash(x)
next(iterator[, default])
chr(i), ord(c)
eval(expr), exec(code)
input(), print()

```

```python
def f(*args, **kwargs): pass
f(1, 2, a=3, b=4)

*args → tuple of extra positional args
**kwargs → dict of extra keyword args

```

math

```
import math

math.sqrt(x)
math.ceil(x), math.floor(x)
math.factorial(x)
math.pow(x, y), math.exp(x)
math.log(x[, base]), math.log10(x), math.log2(x)
math.sin(x), math.cos(x), math.tan(x)
math.degrees(x), math.radians(x)
math.pi, math.e

```

os

```
import os

os.getcwd()
os.chdir(path)
os.listdir(path)
os.mkdir("dir"), os.makedirs("a/b/c")
os.remove("file.txt"), os.rmdir("dir")
os.rename("a.txt", "b.txt")
os.path.exists(path)
os.path.isfile(path), os.path.isdir(path)
os.path.join("a", "b")

```

sys

```
import sys

sys.argv                        # cmd args
sys.exit()
sys.path                        # module search path
sys.stdout.write("hi\n")
```

random

```
import random

random.random()                 # float [0, 1)
random.randint(a, b)            # int [a, b]
random.uniform(a, b)            # float [a, b]
random.choice(seq)
random.choices(seq, k=3)
random.sample(seq, k)
random.shuffle(seq)
```

`datetime`

```
from datetime import datetime, timedelta
from time import time

time()
datetime.now()
datetime.today()
datetime.strptime("2024-01-01", "%Y-%m-%d")
datetime.strftime("%d %b %Y")

delta = timedelta(days=5)
datetime.now() + delta

```

file io

```
with open("file.txt", "r") as f:
    data = f.read()

with open("file.txt", "w") as f:
    f.write("Hello\n")

with open("file.txt", "a") as f:
    f.write("Append\n")

f.readlines()            # returns list of lines
f.readline()             # single line
```

exceptions

```
try:
    risky_code()
except ValueError as e:
    print(e)
except (TypeError, ZeroDivisionError):
    pass
else:
    print("No errors")
finally:
    print("Always")

raise ValueError("bad input")

```

OOP

```python
class Person:
    def __init__(self, name):        # constructor
        self.name = name

    def greet(self):
        print("Hello", self.name)

    @staticmethod
    def static_method():
        print("Static")

    @classmethod
    def from_string(cls, s):
        return cls(s)

p = Person("Alice")
p.greet()

```

Numpy

Tips to mind in keep

1. Start from rightmost (-1-th) dimension; if 1 expand to meet the others, else they're equal. Otherwise invalid
2. When doing `data[:, None, :] - centers[None, :, :]` the a_{ij}-th element will be formed using i-th from `data` and j-th in `centers` ie, order of indices is LITERALLY order of indices
3. When applying an aggregate function (like `sum, mean, median`) along `axis=N` that axis/dimension is lost

Here's a **reorganized NumPy cheat-sheet** in **tabular form**, with **syntax**, **signature**, **description**, and **examples**—ideal for quick exam revision.

---

1. Array Creation

Function	Signature	Description	Example
`np.array`	`np.array(object, dtype=None, copy=True, order='K', subok=False, ndmin=0)`	Create array from any sequence	`np.array([1,2,3], dtype=float)` → `array([1.,2.,3.])`
`np.arange`	`np.arange([start,] stop[, step,], dtype=None)`	Evenly spaced values in `[start,stop)`	`np.arange(0,1,0.2)` → `array([0. ,0.2,0.4,0.6,0.8])`
`np.linspace`	`np.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None)`	`num` samples over `[start,stop]`	`np.linspace(0,1,5)` → `array([0,0.25,0.5,0.75,1])`
`np.zeros`	`np.zeros(shape, dtype=float, order='C')`	Array of zeros	`np.zeros((2,3),int)` → `[[0,0,0], [0,0,0]]`
`np.ones`	`np.ones(shape, dtype=float, order='C')`	Array of ones	`np.ones(4,int)` → `array([1,1,1,1])`
`np.full`	`np.full(shape, fill_value, dtype=None, order='C')`	Array filled with `fill_value`	`np.full((2,2), np.inf)` → `[[inf,inf],[inf,inf]]`

Function	Signature	Description	Example
`np.eye`	`np.eye(N, M=None, k=0, dtype=float)`	2-D identity matrix	`np.eye(3,int)` → `[[1,0,0], [0,1,0],[0,0,1]]`
`np.empty`	`np.empty(shape, dtype=float, order='C')`	Uninitialized array (fast)	`np.empty(3)` → garbage values

---

2. ndarray Attributes & Reshaping

Attribute / Method	Signature	Description	Example
`a.shape`	—	Tuple of array dimensions	`a=np.zeros((2,3)); a.shape` → `(2, 3)`
`a.ndim`	—	Number of axes	`a.ndim` → `2`
`a.size`	—	Total number of elements	`a.size` → `6`
`a.dtype`	—	Data type of elements	`a.dtype` → `dtype('int64')`
`a.reshape`	`a.reshape(newshape, order='C')`	Change view shape	`a.reshape(3,2)`
`a.ravel`	`a.ravel([order])`	Flatten to 1-D view	`a.ravel()`
`a.flatten`	`a.flatten([order])`	Flatten to 1-D copy	`a.flatten()`
`a.astype`	`a.astype(dtype, order='K', casting='unsafe')`	Cast to new dtype	`a.astype(float)`
`a.T`	—	Transpose (swap axes)	`a.T`
`a.copy`	`a.copy([order])`	Deep copy	`b = a.copy()`

---

3. Indexing & Slicing

Technique	Syntax	Description	Example
Basic slicing	`a[start:stop:step]`	Similar to Python lists	`a = np.arange(9).reshape(3,3); a[0,1:3]` → `[1,2]`
Integer indexing	`a[i, j]`	Single element	`a[1,2]` → `5`
Boolean mask	`mask = a > 0; a[mask]`	Select elements satisfying condition	`a[a%2==0]` → evens
Fancy indexing	`a[[i1,i2],[j1,j2]]`	Pick multiple via index arrays	`a[[0,2],[1,2]]`
`np.where`	`np.where(cond)`	Returns indices where `cond` is True	`np.where(a>4)` → `(array([1,2]), array([1,0]))`

---

4. Universal Functions & Broadcasting

Function / Concept	Signature	Description	Example
`np.add`, `np.multiply`, `np.sin`, …	e.g. `np.add(x, y)`	Element-wise arithmetic/ufuncs	`np.add([1,2],[3,4])` → `[4,6]`
Broadcasting rules	—	Auto-expand dims of size 1	`x = [1,2,3]; y = [[10],[20]]; x+y` → `[[11,12,13],[21,22,23]]`

---

5. Aggregation & Statistics

Function	Signature	Description	Example
`np.sum`	`np.sum(a, axis=None, dtype=None, keepdims=False)`	Sum of elements	`np.sum([[1,2],[3,4]])` → `10`
`np.mean`	`np.mean(a, axis=None, dtype=None, keepdims=False)`	Arithmetic mean	`np.mean([1,2,3])` → `2.0`
`np.min`, `np.max`	`np.min(a, axis=None)`, `np.max(a, axis=None)`	Minimum / maximum	`np.min([5,2,9])` → `2`
`np.std`, `np.var`	`np.std(a, axis=None, ddof=0)`, `np.var(a, axis=None, ddof=0)`	Standard deviation / variance	`np.std([1,2,3], ddof=1)`
`np.argmin`, `np.argmax`	`np.argmin(a, axis=None)`, `np.argmax(a, axis=None)`	Index of min / max	`np.argmax([1,5,2])` → `1`

---

6. Linear Algebra (`numpy.linalg`)

Function	Signature	Description	Example
`np.dot`	`np.dot(a, b)`	Dot product	`np.dot([1,2],[3,4])` → `11`
`a @ b`	—	Matrix multiplication	`A @ B`
`np.linalg.inv`	`np.linalg.inv(a)`	Inverse of square matrix	`np.linalg.inv([[1,2],[3,4]])`
`np.linalg.det`	`np.linalg.det(a)`	Determinant	`np.linalg.det([[1,2],[3,4]])`
`np.linalg.eig`	`np.linalg.eig(a)`	Eigenvalues & vectors	`np.linalg.eig([[1,2],[2,1]])`

---

7. Random Sampling (`numpy.random`)

Function	Signature	Description	Example
`rng = np.random.default_rng(seed)`	—	Create a Generator instance	`rng = default_rng(123)`
`rng.random`	`rng.random(size=None, dtype=float, out=None, *, method='PCG64')`	Uniform [0,1) floats	`rng.random(5)` → array of 5 floats

Function	Signature	Description	Example
`rng.integers`	`rng.integers(low, high=None, size=None, dtype=int)`	Random ints in `[low, high)`	`rng.integers(0,10, size=3)`
`rng.normal`	`rng.normal(loc=0.0, scale=1.0, size=None)`	Draw samples from normal distribution	`rng.normal(0,1, size=4)`
`rng.choice`	`rng.choice(a, size=None, replace=True, p=None)`	Random sample from `a`	`rng.choice([10,20,30], size=2)`
`np.random.rand`	`np.random.rand(d0, d1, …)`	Convenience uniform random	`np.random.rand(2,3)`

---

8. I/O Routines

Function	Signature	Description	Example
`np.save`	`np.save(file, arr, allow_pickle=True, fix_imports=True)`	Save array to binary `.npy` file	`np.save('a.npy', A)`
`np.load`	`np.load(file, mmap_mode=None, allow_pickle=False)`	Load `.npy` or `.npz` file	`A = np.load('a.npy')`
`np.savez`	`np.savez(file, *args, **kwds)`	Save multiple arrays to `.npz` archive	`np.savez('multi.npz', x=A, y=B)`
`np.savetxt`	`np.savetxt(fname, X, fmt='%.18e', delimiter=' ', newline='\n')`	Save 1D/2D array to text file	`np.savetxt('data.txt', A, fmt='%d')`
`np.loadtxt`	`np.loadtxt(fname, dtype=float, delimiter=None)`	Load 1D/2D data from text	`B = np.loadtxt('data.txt', delimiter=',')`

Keep this sheet handy—you can scan each table for syntax, signature, and a quick example under pressure. Good luck!

mpl

Here is a **well-organized Matplotlib `pyplot` cheat sheet**, with **syntax, call signatures, descriptions**, and **examples**, focusing on:

- `plot()`, `scatter()`, `bar()`, `hist()`
- **Plot formatting** (labels, legends, colors, markers, line styles)
- **Subplots**
- **Saving figures**

📈 Basic Plotting (`plt.plot`)

Function	Signature	Description	Example
`plt.plot`	`plt.plot(x, y, fmt='', **kwargs)`	Line plot	`plt.plot(x, y, 'ro--')` → red circle dotted line

Function	Signature	Description	Example
`plt.title`	`plt.title(label, fontsize=..., loc=...)`	Plot title	`plt.title("My Plot")`
`plt.xlabel`	`plt.xlabel(label, fontsize=...)`	X-axis label	`plt.xlabel("Time")`
`plt.ylabel`	`plt.ylabel(label, fontsize=...)`	Y-axis label	`plt.ylabel("Value")`
`plt.grid`	`plt.grid(True/False, which='both', axis='both')`	Toggle grid	`plt.grid(True)`
`plt.legend`	`plt.legend([labels], loc='best')`	Show legend	`plt.legend(["Train", "Test"])`

Example:

```
import matplotlib.pyplot as plt
x = [0, 1, 2, 3]
y = [0, 1, 4, 9]
plt.plot(x, y, color='blue', linestyle='--', marker='o', label='x squared')
plt.title("Quadratic Plot")
plt.xlabel("X")
plt.ylabel("Y")
plt.grid(True)
plt.legend()
plt.show()
```

📊 Bar Plot (`plt.bar`)

Function	Signature	Description	Example
`plt.bar`	`plt.bar(x, height, width=0.8, align='center', **kwargs)`	Vertical bars	`plt.bar(['A','B'], [5,3])`
`plt.barh`	`plt.barh(y, width, height=0.8, align='center', **kwargs)`	Horizontal bars	`plt.barh(['A','B'], [5,3])`

Example:

```
plt.bar(['Apples', 'Bananas', 'Cherries'], [30, 25, 40], color='green')
plt.ylabel("Quantity")
plt.title("Fruit Sales")
plt.show()
```

● Scatter Plot (`plt.scatter`)

Function	Signature	Description	Example
`plt.scatter`	`plt.scatter(x, y, s=20, c=None, marker='o', alpha=1.0, **kwargs)`	Scatter plot	`plt.scatter(x, y, color='r', marker='x')`

Example with size and color:

```
x = [1, 2, 3, 4]
y = [10, 20, 25, 30]
sizes = [20, 50, 80, 200]
colors = ['red', 'blue', 'green', 'orange']
plt.scatter(x, y, s=sizes, c=colors, alpha=0.6)
plt.title("Scatter Plot Example")
plt.show()
```

📉 Histogram (`plt.hist`)

Function	Signature	Description	Example
`plt.hist`	`plt.hist(x, bins=10, range=None, density=False, cumulative=False)`	Histogram of array data	`plt.hist(data, bins=5, color='gray')`

Example:

```
import numpy as np
data = np.random.normal(0, 1, size=1000)
plt.hist(data, bins=30, color='skyblue', edgecolor='black')
plt.title("Normal Distribution")
plt.show()
```

🎨 Plot Formatting Options

Parameter	Usage	Values/Examples
`color` / `c`	Line/marker/bar color	`'blue'`, `'#ff0000'`, `'r'`
`linestyle` / `ls`	Line style	`'solid'`, `'--'`, `':'`, `'-.'`
`linewidth` / `lw`	Line width	`1.0`, `2`, etc.
`marker`	Point marker	`'o'`, `'s'`, `'^'`, `'x'`, `'.'`
`alpha`	Transparency	0.0 (transparent) to 1.0 (opaque)
`label`	Legend label	`label="My Line"`
`markersize` / `ms`	Marker size	`5`, `10`, etc.

🖼️ Subplots and Layout

Function	Signature	Description	Example
`plt.subplot`	`plt.subplot(nrows, ncols, index)`	Single subplot	`plt.subplot(2,2,1)`
`plt.subplots`	`plt.subplots(nrows, ncols, figsize=(w,h), sharex=False, sharey=False)`	Grid of subplots	`fig, ax = plt.subplots(2,1)`

Function	Signature	Description	Example
`plt.tight_layout`	—	Auto layout spacing	`plt.tight_layout()`

Example:

```
fig, axs = plt.subplots(1, 2, figsize=(10, 4))
axs[0].plot([1,2,3], [1,4,9])
axs[1].bar(['A','B'], [5,3])
plt.tight_layout()
plt.show()
```

💾 Saving Figures

Function	Signature	Description	Example
`plt.savefig`	`plt.savefig(fname, dpi=..., format=..., bbox_inches=...)`	Save current figure	`plt.savefig('plot.png', dpi=300)`

- Use `bbox_inches='tight'` to avoid clipped labels.
- Common formats: `.png`, `.pdf`, `.svg`, `.jpg`

Let me know if you want a **styling cheat sheet** (Seaborn or LaTeX text, grid style, themes) or **interactive plotting** tips (`mplcursors`, `widgets`, etc).